

ATmega2560 Bare-Metal Driver Library

GPIO · LED · Switch · Timer · 7-Segment · IR · Ultrasonic · PWM · ADC

Features

- Nine register-level drivers: GPIO, LED, Switch, Timer, 7-Segment, IR, Ultrasonic, PWM, ADC
- Plain C for avr-gcc, no avr-libc headers and no Arduino code
- Every register accessed by its datasheet data-space address
- General drivers: pins and options are passed to init, not hard-coded
- Several LEDs, switches, IR and ultrasonic sensors through ids
- Safe on wrong arguments: wrong port, pin, id or channel is ignored
- Interrupt handlers without avr-libc (`__vector_17`, `__vector_21`)
- 16-bit registers accessed in datasheet order (high byte first on write, low byte first on read)
- Small footprint: about 3.7 KB flash for all nine drivers
- Builds with -Wall -Wextra and zero warnings

Summary

Target	ATmega2560 (Arduino Mega 2560)
Clock	16 MHz
Language	C (beginner style, no structs)
Toolchain	avr-gcc, avrdude
Timers used	Timer0, Timer1, Timer4, Timer5
Interrupts used	TIMER0 COMPA, TIMER1 COMPA
Flash (all drivers)	about 3.7 KB
Status	Builds; not yet tested on hardware

Description

A set of small, general drivers for the ATmega2560 microcontroller on the Arduino Mega 2560 board. Each driver writes the chip registers directly and gives the application a short, simple API. The drivers were written for a smart-parking project, but contain nothing project specific: the wiring of a project lives only in its own `board.h`.

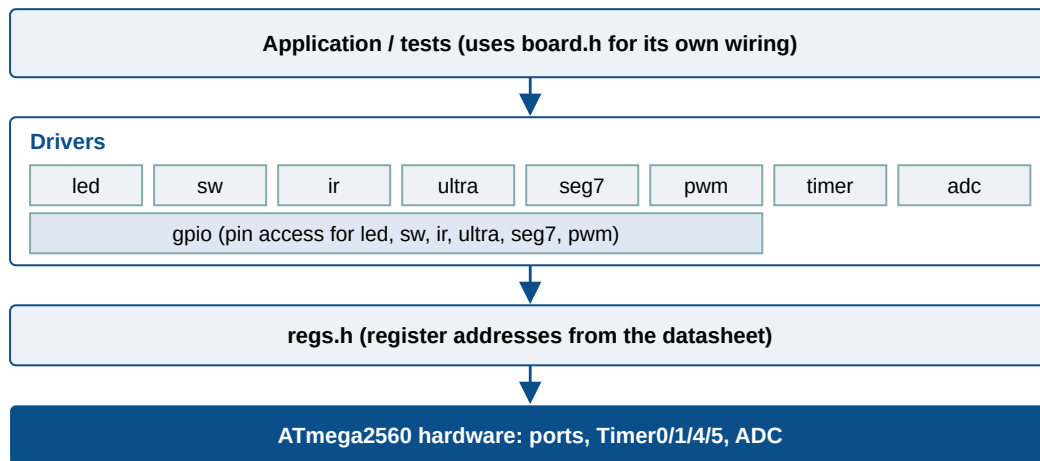
Contents

1	Overview	3
2	GPIO Driver	5
3	LED Driver	8
4	Switch Driver	10
5	Timer Driver	12
6	7-Segment Display Driver	14
7	IR Sensor Driver	17
8	Ultrasonic Sensor Driver	19
9	PWM Driver	22
10	ADC Driver	24
11	Register Summary	26
12	Interrupts and 16-bit Access	28
13	Memory Footprint and Build	29
14	Reference Wiring (Smart Parking Project)	30
15	Revision History	31

1 Overview

1.1 Architecture

The library is built in layers. Application code only calls driver functions. Drivers that work with pins use the GPIO driver; drivers that own a peripheral (timers, ADC) write its registers directly. All registers come from `regs.h`, where each one is defined by its data-space address from the ATmega2560 datasheet.



1.2 Design rules

- **General:** no driver includes `board.h`. Pins and options are parameters of the init functions.
- **Ids:** drivers for parts that can appear more than once (LED, switch, IR, ultrasonic) return an id from init. Later calls take that id. A wrong pin or a full table makes init return `..._NONE` (255).
- **Safe arguments:** a wrong port, pin, id, channel or value never writes to a random register. The function does nothing or returns 0 (or the documented error value).
- **No libraries:** no avr-libc headers. avr-gcc still links its own start-up code (stack set-up, vector table).
- **Style:** plain C with `unsigned char/int/long`, `if/else`, switch and arrays. No structs or function pointers.

1.3 Hardware resources

Resource	Used by	Setup	Result
Timer0	seg7	CTC, /256, OCR0A = 124	Interrupt every 2 ms (digit refresh)
Timer1	timer (and sw)	CTC, /64, OCR1A = 249	Interrupt every 1 ms (millis)
Timer4	pwm	Fast PWM mode 14, /8, TOP = ICR4	PWM on OC4A/B/C, default 2 kHz
Timer5	ultra	Normal mode, /8	Free-running, 0.5 us per tick
ADC	adc	AVcc ref, /128	125 kHz ADC clock
Timer2, Timer3	-	-	Free for the application

1.4 Driver dependencies

Driver	Depends on	Turns global interrupts on
gpio	-	No
led	gpio	No
sw	gpio, timer	Yes (through timer_init)
timer	-	Yes

Driver	Depends on	Turns global interrupts on
seg7	gpio	Yes
ir	gpio	No
ultra	gpio	No
pwm	gpio	No
adc	-	No

2 GPIO Driver

2.1 Description

Single-pin control for all eleven I/O ports of the ATmega2560 (A to L, there is no port I). Every other driver that touches a pin goes through this driver.

2.2 Features

- Ports 'A'..'L' (no 'I'), pins 0..7
- Output, floating input or input with internal pull-up
- Set, clear, read and toggle a single pin
- Wrong port or pin is ignored (no crash, no write)
- No init needed: all pins are inputs after reset

2.3 Summary

Item	Value
Files	drivers/gpio/gpio.c, gpio.h
Depends on	None (uses regs.h only)
Registers	PINx, DDRx, PORTx for x = A..L
Code size (flash)	384 bytes

2.4 Constants

Name	Value	Meaning
GPIO_IN	0	Input, pull-up off (floating)
GPIO_OUT	1	Output
GPIO_IN_PULLUP	2	Input with internal pull-up on

2.5 Function reference

```
void gpio_dir(char port, unsigned char pin, unsigned char dir);
```

Sets the direction of one pin. For GPIO_IN the pull-up is switched off, for GPIO_IN_PULLUP it is switched on.

Parameter	Description
port	'A'..'L' (no 'I')
pin	0..7
dir	GPIO_IN, GPIO_OUT or GPIO_IN_PULLUP

Returns: Nothing.

Note: Any dir value other than GPIO_OUT or GPIO_IN_PULLUP is treated as GPIO_IN.

```
void gpio_set(char port, unsigned char pin);
```

Drives an output pin high (5 V). On an input pin this turns the pull-up on.

Parameter	Description
port	'A'..'L'
pin	0..7

Returns: Nothing.

```
void gpio_clear(char port, unsigned char pin);
```

Drives an output pin low (0 V). On an input pin this turns the pull-up off.

Parameter	Description
port	'A'..'L'
pin	0..7

Returns: Nothing.

```
unsigned char gpio_get(char port, unsigned char pin);
```

Reads the real voltage level on the pin from the PINx register. Works for inputs and outputs.

Parameter	Description
port	'A'..'L'
pin	0..7

Returns: 1 if the pin is high, 0 if low. Returns 0 for a wrong port or pin.

```
void gpio_invert(char port, unsigned char pin);
```

Flips an output pin (high to low or low to high) using `PORTx ^= bit`.

Parameter	Description
port	'A'..'L'
pin	0..7

Returns: Nothing.

2.6 Characteristics

Parameter	Value
Ports supported	11 (A, B, C, D, E, F, G, H, J, K, L)
Port access	A..G: I/O space 0x20..0x34, H..L: extended I/O 0x100..0x10B
Interrupt use	None

2.7 Usage

Example

```
gpio_dir('B', 7, GPIO_OUT);      /* on-board LED, D13 */
gpio_set('B', 7);                /* LED on */

gpio_dir('E', 4, GPIO_IN_PULLUP); /* button to GND on D2 */
if (gpio_get('E', 4) == 0)
{
    /* button pressed */
}
```

3 LED Driver

3.1 Description

Drives LEDs on any pin. Each call to `led_init` adds one LED and returns an id. An LED is on when its pin is high (LED from pin through a resistor to GND).

3.2 Features

- Up to LED_MAX (8) LEDs on any port/pin
- On, off, toggle one LED, or switch all LEDs off
- LEDs start off after `led_init`
- Wrong id is ignored

3.3 Summary

Item	Value
Files	<code>drivers/led/led.c</code> , <code>led.h</code>
Depends on	<code>gpio</code>
Registers	None directly (through <code>gpio</code>)
Code size (flash)	214 bytes

3.4 Constants

Name	Value	Meaning
LED_MAX	8	Maximum number of LEDs
LED_NONE	255	Returned by <code>led_init</code> when it fails

3.5 Function reference

```
unsigned char led_init(char port, unsigned char pin);
```

Adds an LED: makes the pin an output and switches the LED off.

Parameter	Description
<code>port</code>	'A'..'L' (no 'I')
<code>pin</code>	0..7

Returns: The LED id (0..7), or LED_NONE if the port/pin is wrong or 8 LEDs are already added.

```
void led_on(unsigned char id);
```

Switches the LED on (pin high).

Parameter	Description
<code>id</code>	Value returned by <code>led_init</code>

Returns: Nothing.

```
void led_off(unsigned char id);
```

Switches the LED off (pin low).

Parameter	Description
id	Value returned by led_init

Returns: Nothing.

```
void led_toggle(unsigned char id);
```

Flips the LED.

Parameter	Description
id	Value returned by led_init

Returns: Nothing.

```
void led_all_off(void);
```

Switches every added LED off.

Returns: Nothing.

3.6 Characteristics

Parameter	Value
RAM used	17 bytes (port and pin table for 8 LEDs + count)
Active level	High = LED on

3.7 Usage

Example

```
unsigned char stop_led;

stop_led = led_init('A', 3);  /* LED on PA3 (D25) */
led_on(stop_led);
led_toggle(stop_led);         /* now off */
```

4 Switch Driver

4.1 Description

Reads push buttons on any pin. `sw_is_pressed` gives the raw state; `sw_was_pressed` gives one event per press with a 20 ms software debounce based on `timer_millis`.

4.2 Features

- Up to `SW_MAX` (4) switches on any port/pin
- Active-low (switch to GND, internal pull-up) or active-high (switch to 5 V)
- Raw state or debounced press event
- 20 ms debounce, kept separately for each switch
- Starts the timer driver by itself

4.3 Summary

Item	Value
Files	<code>drivers/sw/sw.c</code> , <code>sw.h</code>
Depends on	gpio, timer
Registers	None directly (through gpio and timer)
Code size (flash)	322 bytes

4.4 Constants

Name	Value	Meaning
<code>SW_MAX</code>	4	Maximum number of switches
<code>SW_NONE</code>	255	Returned by <code>sw_init</code> when it fails
<code>SW_ACTIVE_LOW</code>	1	Switch to GND, internal pull-up is turned on
<code>SW_ACTIVE_HIGH</code>	0	Switch to 5 V, an external pull-down resistor is needed

4.5 Function reference

```
unsigned char sw_init(char port, unsigned char pin, unsigned char active_low);
```

Adds a switch and sets its pin as input (with pull-up for `SW_ACTIVE_LOW`). Also calls `timer_init()`, which turns global interrupts on.

Parameter	Description
<code>port</code>	'A'..'L' (no 'I')
<code>pin</code>	0..7
<code>active_low</code>	<code>SW_ACTIVE_LOW</code> or <code>SW_ACTIVE_HIGH</code>

Returns: The switch id (0..3), or `SW_NONE` if the port/pin is wrong or 4 switches are already added.

```
unsigned char sw_is_pressed(unsigned char id);
```

Reads the switch right now, without debounce.

Parameter	Description
<code>id</code>	Value returned by <code>sw_init</code>

Returns: 1 while pressed, 0 if not pressed or wrong id.

Note: Contact bounce can give several changes within a few milliseconds.

```
unsigned char sw_was_pressed(unsigned char id);
```

Debounced press event. The raw level must stay the same for 20 ms before it is accepted.

Parameter	Description
<code>id</code>	Value returned by <code>sw_init</code>

Returns: 1 exactly once for each new press, else 0.

Note: Call it often (every few milliseconds) from the main loop, or presses can be missed.

4.6 Characteristics

Parameter	Value
Debounce time	20 ms
RAM used	37 bytes for 4 switches (pins, type, debounce state)

4.7 Usage

Example

```
unsigned char key;

key = sw_init('E', 4, SW_ACTIVE_LOW);  /* button on PE4 (D2) to GND */

while (1)
{
    if (sw_was_pressed(key))
    {
        /* runs once per press */
    }
}
```

5 Timer Driver

5.1 Description

System tick of 1 ms made with Timer1 in CTC mode. Gives a millisecond counter, a blocking delay and a non-blocking time check.

5.2 Features

- 1 ms interrupt from Timer1 (CTC, prescaler 64, OCR1A = 249)
- 32-bit millisecond counter, read safely with interrupts briefly off
- Blocking delay and non-blocking elapsed-time check
- Rollover safe (counter wraps after about 49.7 days)
- `timer_init` can be called more than once

5.3 Summary

Item	Value
Files	<code>drivers/timer/timer.c</code> , <code>timer.h</code>
Depends on	None (uses <code>regs.h</code> only)
Registers	TCCR1A, TCCR1B, OCR1AH/L, TIMSK1, SREG
Code size (flash)	380 bytes

5.4 Function reference

```
void timer_init(void);
```

Starts Timer1 with a 1 ms compare-match interrupt and turns global interrupts on (SREG bit I). A second call does nothing, so the count is kept.

Returns: Nothing.

```
unsigned long timer_millis(void);
```

Milliseconds since `timer_init`. The 4-byte counter is copied with interrupts off, then SREG is restored.

Returns: Milliseconds, 0..4294967295.

Note: An 8-bit CPU needs 4 reads for a 32-bit value, so the copy is protected from the interrupt.

```
void timer_delay_ms(unsigned long ms);
```

Waits in a loop until `ms` milliseconds have passed. Interrupts keep running.

Parameter	Description
<code>ms</code>	Milliseconds to wait

Returns: Nothing.

Note: Needs `timer_init` first, otherwise it waits forever.

```
unsigned char timer_elapsed(unsigned long start, unsigned long ms);
```

Non-blocking check: has ms milliseconds passed since start?

Parameter	Description
start	An earlier value of timer_millis()
ms	Time span in milliseconds

Returns: 1 if passed, else 0.

5.5 Characteristics

Parameter	Value
Tick period	1 ms (16 MHz / 64 / 250)
Counter width / wrap	32 bit / about 49.7 days
Interrupt	TIMER1 COMPA, vector No. 18, handler __vector_17
RAM used	5 bytes

5.6 Usage

Example

```
unsigned long last;

timer_init();
last = timer_millis();

while (1)
{
    if (timer_elapsed(last, 500))
    {
        last = last + 500;
        /* runs every 500 ms, loop never blocks */
    }
}
```

6 7-Segment Display Driver

6.1 Description

Drives a 2-digit multiplexed 7-segment display. Segments a..g and dp sit on pins 0..7 of one port; each digit has its own enable pin. Timer0 shows one digit every 2 ms, so the main program only sets what to show.

6.2 Features

- Any segment port and any two digit-enable pins
- Common cathode or common anode display
- Digit enable active high (transistor) or active low
- Numbers 0..99 (leading zero blanked), hex digits 0..F, dash, blank
- Decimal point per digit and raw segment patterns
- Refresh by interrupt, no CPU time needed in the main loop

6.3 Summary

Item	Value
Files	drivers/seg7/seg7.c, seg7.h
Depends on	gpio
Registers	TCCR0A, TCCR0B, OCR0A, TIMSK0, SREG
Code size (flash)	696 bytes

6.4 Constants

Name	Value	Meaning
SEG7_CATHODE	0	Common cathode: segment on = pin high
SEG7_ANODE	1	Common anode: segment on = pin low

6.5 Function reference

```
void seg7_init(char seg_port, char d1_port, unsigned char d1_pin, char d2_port, unsigned char d2_pin, unsigned char type, unsigned char digit_on_level);
```

Saves the pins and options, makes all pins outputs, blanks the display, starts Timer0 (CTC, /256, OCR0A = 124) and turns global interrupts on.

Parameter	Description
seg_port	Port with segments a..g, dp on pins 0..7
d1_port, d1_pin	Enable pin of the left digit (pos 0, tens)
d2_port, d2_pin	Enable pin of the right digit (pos 1, ones)
type	SEG7_CATHODE or SEG7_ANODE
digit_on_level	Pin level that turns a digit on: 1 through an NPN transistor, 0 for a common-cathode pin driven directly

Returns: Nothing. With a wrong port or pin the display is not started.

```
void seg7_show_number(unsigned char num);
```

Shows 0..99. For 0..9 the left digit is blank. Above 99 it shows a dash on both digits.

Parameter	Description
num	0..99

Returns: Nothing.

Note: Decimal points are not changed.

```
void seg7_show_digit(unsigned char pos, unsigned char value);
```

Shows one hex digit on one position.

Parameter	Description
pos	0 = left, 1 = right
value	0..15 (0-9, A, b, C, d, E, F)

Returns: Nothing. Wrong pos or value is ignored.

```
void seg7_show_dash(void);
```

Shows a dash (segment g) on both digits.

Returns: Nothing.

```
void seg7_blank(void);
```

Turns both digits and both decimal points off.

Returns: Nothing.

```
void seg7_set_dp(unsigned char pos, unsigned char on);
```

Turns the decimal point of one digit on or off.

Parameter	Description
pos	0 or 1
on	1 = on, 0 = off

Returns: Nothing.

```
void seg7_show_raw(unsigned char pos, unsigned char pattern);
```

Shows any segment pattern on one digit.

Parameter	Description
pos	0 or 1
pattern	bit0 = a, bit1 = b ... bit6 = g, bit7 = dp

Returns: Nothing.

6.6 Characteristics

Parameter	Value
Refresh	2 ms per digit, each digit 250 Hz at 50 % duty
Interrupt	TIMER0 COMPA, vector No. 22, handler __vector_21
Characters	0-9, A, b, C, d, E, F, dash, blank
RAM used	12 bytes + 18-byte segment table

6.7 Usage

Example

```
seg7_init('C', 'G', 0, 'G', 1, SEG7_CATHODE, 1);

seg7_show_number(42); /* "42" */
seg7_show_digit(0, 10); /* left digit "A" */
seg7_set_dp(1, 1); /* dot on right digit */
```

7 IR Sensor Driver

7.1 Description

Reads digital IR obstacle-sensor modules on any pin. `ir_is_detected` reads the sensor three times about 1 ms apart and only reports an object when all three agree, which filters short glitches.

7.2 Features

- Up to `IR_MAX` (4) sensors on any port/pin
- Active-low (most modules) or active-high output
- Raw pin read or filtered detection (3 samples)
- Internal pull-up on, so an unplugged pin does not float

7.3 Summary

Item	Value
Files	<code>drivers/ir/ir.c</code> , <code>ir.h</code>
Depends on	<code>gpio</code>
Registers	None directly (through <code>gpio</code>)
Code size (flash)	232 bytes

7.4 Constants

Name	Value	Meaning
<code>IR_MAX</code>	4	Maximum number of sensors
<code>IR_NONE</code>	255	Returned by <code>ir_init</code> when it fails
<code>IR_ACTIVE_LOW</code>	1	Output is 0 when an object is seen (most modules)
<code>IR_ACTIVE_HIGH</code>	0	Output is 1 when an object is seen

7.5 Function reference

```
unsigned char ir_init(char port, unsigned char pin, unsigned char active_low);
```

Adds a sensor and sets its pin as input with pull-up.

Parameter	Description
<code>port</code>	'A'..'L' (no 'I')
<code>pin</code>	0..7
<code>active_low</code>	<code>IR_ACTIVE_LOW</code> or <code>IR_ACTIVE_HIGH</code>

Returns: The sensor id (0..3), or `IR_NONE` if the port/pin is wrong or 4 sensors are already added.

```
unsigned char ir_read_raw(unsigned char id);
```

Reads the sensor output pin right now.

Parameter	Description
id	Value returned by ir_init

Returns: Pin level 0 or 1 (0 for a wrong id).

```
unsigned char ir_is_detected(unsigned char id);
```

Filtered detection: three reads about 1 ms apart must all see an object.

Parameter	Description
id	Value returned by ir_init

Returns: 1 if an object is seen in all three reads, else 0.

Note: Takes about 2 ms when an object is seen; returns at once on the first clear read. The wait is a busy loop, so the IR driver does not need a timer.

7.6 Characteristics

Parameter	Value
Samples per detection	3, about 0.95 ms apart (busy loop, 19 cycles x 800)
Detection time	about 1.9 ms (object), less than 10 us (no object)
RAM used	13 bytes

7.7 Usage

Example

```
unsigned char slot;

slot = ir_init('L', 3, IR_ACTIVE_LOW);  /* sensor OUT on PL3 (D46) */

if (ir_is_detected(slot))
{
    /* something is in front of the sensor */
}
```

8 Ultrasonic Sensor Driver

8.1 Description

Measures distance with HC-SR04 type sensors on any pins. Timer5 runs freely at 0.5 us per tick and is used for the 10 us trigger pulse, the echo time and both 30 ms timeouts.

8.2 Features

- Up to ULTRA_MAX (4) sensors, trigger and echo on any pins
- Echo time in microseconds or distance in centimetres
- Averaging of 1..8 readings with timeouts skipped
- 30 ms timeout while waiting for the echo to start and to end
- No interrupts used, measurement is done by polling

8.3 Summary

Item	Value
Files	drivers/ultra/ultra.c, ultra.h
Depends on	gpio
Registers	TCCR5A, TCCR5B, TCNT5H/L
Code size (flash)	648 bytes

8.4 Constants

Name	Value	Meaning
ULTRA_MAX	4	Maximum number of sensors
ULTRA_NONE	255	Returned by ultra_init when it fails
ULTRA_NO_ECHO	999	cm functions: no echo / timeout / wrong id
ULTRA_NO_ECHO_US	65535	ultra_get_us: no echo / timeout / wrong id

8.5 Function reference

```
unsigned char ultra_init(char trig_port, unsigned char trig_pin, char echo_port, unsigned char echo_pin);
```

Adds a sensor: trigger pin output (low), echo pin input. Starts Timer5 in normal mode with prescaler 8.

Parameter	Description
trig_port, trig_pin	Pin connected to TRIG
echo_port, echo_pin	Pin connected to ECHO

Returns: The sensor id (0..3), or ULTRA_NONE if a port/pin is wrong or 4 sensors are already added.

```
unsigned int ultra_get_us(unsigned char id);
```

Sends a 10 us trigger pulse and measures how long the echo stays high.

Parameter	Description
id	Value returned by ultra_init

Returns: Echo time in microseconds, or ULTRA_NO_ECHO_US on timeout or wrong id.

Note: 999 is not used as the timeout value here because 999 us is a real echo (about 17 cm).

```
unsigned int ultra_get_cm(unsigned char id);
```

Distance in cm = echo time in us / 58.

Parameter	Description
id	Value returned by ultra_init

Returns: Distance in cm, or ULTRA_NO_ECHO on timeout or wrong id.

```
unsigned int ultra_get_cm_avg(unsigned char id, unsigned char n);
```

Takes n readings 60 ms apart, skips timeouts and returns the average of the good ones.

Parameter	Description
id	Value returned by ultra_init
n	Number of readings, 1..8

Returns: Average distance in cm, or ULTRA_NO_ECHO if all readings failed or id/n is wrong.

Note: Blocks for up to n x 60 ms plus the readings themselves.

8.6 Characteristics

Parameter	Value
Timer resolution	0.5 us (16 MHz / 8)
Trigger pulse	10 us
Echo timeout	30 ms (start) + 30 ms (end)
Distance resolution	1 cm (integer)
Max time per reading	about 60 ms (both timeouts)
RAM used	17 bytes

8.7 Usage

Example

```
unsigned char rear;
unsigned int cm;

rear = ultra_init('L', 2, 'L', 1);  /* TRIG PL2 (D47), ECHO PL1 (D48) */

cm = ultra_get_cm_avg(rear, 3);
if (cm != ULTRA_NO_ECHO)
{
    /* use cm */
}
```

9 PWM Driver

9.1 Description

Hardware PWM on the three Timer4 outputs. Fast PWM mode 14 with TOP = ICR4, so the frequency can be set freely. All three channels share the same frequency; each has its own duty cycle.

9.2 Features

- Channels 'A' = OC4A = PH3 (D6), 'B' = OC4B = PH4 (D7), 'C' = OC4C = PH5 (D8)
- Frequency 31..65535 Hz, default 2 kHz
- Duty 0..100 % per channel
- Duty is kept when the frequency changes
- Non-inverting output, pin driven low when off

9.3 Summary

Item	Value
Files	drivers/pwm/pwm.c, pwm.h
Depends on	gpio
Registers	TCCR4A, TCCR4B, ICR4H/L, OCR4AH/L, OCR4BH/L, OCR4CH/L
Code size (flash)	520 bytes

9.4 Function reference

```
void pwm_init(char ch);
```

Makes the channel pin an output (low), sets Timer4 up on the first call (mode 14, prescaler 8, 2 kHz) and sets duty 0. The output stays off until `pwm_on`.

Parameter	Description
ch	'A', 'B' or 'C'

Returns: Nothing. A wrong channel is ignored.

```
void pwm_set_freq(unsigned int hz);
```

Sets the PWM frequency for all channels: $ICR4 = 2000000 / hz - 1$. Every channel's duty is applied again.

Parameter	Description
hz	31..65535

Returns: Nothing. Values below 31 are ignored.

Note: All three channels always run at the same frequency.

```
void pwm_set_duty(char ch, unsigned char duty);
```

Sets the duty cycle: $OCR4x = (ICR4 + 1) \times \text{duty} / 100$ (32-bit math).

Parameter	Description
ch	'A', 'B' or 'C'
duty	0..100 percent

Returns: Nothing. Wrong channel or duty above 100 is ignored.

Note: Duty 0 still gives one 0.5 us pulse per period (fast PWM behaviour). Use `pwm_off` for a fully low pin.

```
void pwm_on(char ch);
```

Connects the PWM signal to the pin ($COM4x1 = 1$, non-inverting).

Parameter	Description
ch	'A', 'B' or 'C'

Returns: Nothing.

```
void pwm_off(char ch);
```

Disconnects the PWM signal and drives the pin low.

Parameter	Description
ch	'A', 'B' or 'C'

Returns: Nothing.

9.5 Characteristics

Parameter	Value
Timer clock	2 MHz (16 MHz / 8)
Frequency	31..65535 Hz, $f = 2 \text{ MHz} / (ICR4 + 1)$
Duty steps	$ICR4 + 1$ (1000 steps at 2 kHz, 30 steps at 65 kHz)
RAM used	6 bytes

9.6 Usage

Example

```
pwm_init('A');           /* D6 */
pwm_set_freq(4000);      /* 4 kHz */
pwm_set_duty('A', 50);   /* 50 % */
pwm_on('A');
```

10 ADC Driver

10.1 Description

10-bit analog-to-digital converter with AVcc (5 V) as reference. Single conversions by polling on any of the 16 analog inputs A0..A15.

10.2 Features

- Channels 0..15 (A0..A15), MUX5 handled for 8..15
- ADC clock 125 kHz (16 MHz / 128), inside the 50..200 kHz range for full accuracy
- Averaging of 1..16 readings
- Raw value to millivolts conversion
- One dummy conversion in init (the first one is slow)

10.3 Summary

Item	Value
Files	drivers/adc/adc.c, adc.h
Depends on	None (uses regs.h only)
Registers	ADMUX, ADCSRA, ADCSRB, ADCL, ADCH
Code size (flash)	260 bytes

10.4 Function reference

```
void adc_init(void);
```

Reference AVcc, right-adjusted result, ADC on with prescaler 128, then one dummy conversion.

Returns: Nothing.

```
unsigned int adc_read(unsigned char channel);
```

Selects the channel, starts one conversion (ADSC) and waits for it. ADCL is read before ADCH.

Parameter	Description
channel	0..15 (A0..A15)

Returns: 0..1023, or 0 for a channel above 15.

Note: Reading ADCL first locks the result until ADCH is read, so both bytes come from the same conversion.

```
unsigned int adc_read_avg(unsigned char channel, unsigned char n);
```

Average of n conversions.

Parameter	Description
channel	0..15
n	1..16

Returns: Average 0..1023, or 0 for a wrong channel or n.

```
unsigned int adc_to_mv(unsigned int raw);
```

Converts a raw value to millivolts: $\text{raw} \times 5000 / 1023$ (32-bit math).

Parameter	Description
raw	0..1023

Returns: 0..5000 mV (assumes AVcc is exactly 5.0 V).

10.5 Characteristics

Parameter	Value
Resolution	10 bit, about 4.9 mV per step at 5 V
ADC clock	125 kHz
Conversion time	about 104 us (13 ADC clocks), first one about 200 us
Interrupt use	None

10.6 Usage

Example

```
unsigned int raw;
unsigned int mv;

adc_init();
raw = adc_read_avg(0, 8);    /* pot on A0, 8 readings */
mv = adc_to_mv(raw);        /* 0..5000 */
```

11 Register Summary

All addresses are data-space addresses, taken from the Register Summary in the Atmel ATmega2560 datasheet (document 2549Q-AVR-02/2014). For registers below 0x60 the datasheet shows two numbers, for example 0x04 (0x24) DDRB. The first is the I/O address used only by the IN/OUT instructions; the number in brackets is the memory address, which a C pointer must use. Data address = I/O address + 0x20.

Register	Data address	I/O address	Used by	Datasheet page
PINA / DDRA / PORTA	0x20 / 0x21 / 0x22	0x00-0x02	gpio	96
PINB / DDRB / PORTB	0x23 / 0x24 / 0x25	0x03-0x05	gpio	96
PINC / DDRC / PORTC	0x26 / 0x27 / 0x28	0x06-0x08	gpio	97
PIND / DDRD / PORTD	0x29 / 0x2A / 0x2B	0x09-0x0B	gpio	97
PINE / DDRE / PORTE	0x2C / 0x2D / 0x2E	0x0C-0x0E	gpio	97-98
PINF / DDRF / PORTF	0x2F / 0x30 / 0x31	0x0F-0x11	gpio	97-98
PING / DDRG / PORTG	0x32 / 0x33 / 0x34	0x12-0x14	gpio	98
PINH / DDRH / PORTH	0x100 / 0x101 / 0x102	-	gpio	98-99
PINJ / DDRJ / PORTJ	0x103 / 0x104 / 0x105	-	gpio	99
PINK / DDRK / PORTK	0x106 / 0x107 / 0x108	-	gpio	99
PINL / DDRL / PORTL	0x109 / 0x10A / 0x10B	-	gpio	100
TCCR0A / TCCR0B	0x44 / 0x45	0x24 / 0x25	seg7	126, 129
OCR0A	0x47	0x27	seg7	130
SREG	0x5F	0x3F	timer, seg7	13
TIMSK0	0x6E	-	seg7	131
TIMSK1	0x6F	-	timer	161
ADCL / ADCH	0x78 / 0x79	-	adc	286
ADCSRA	0x7A	-	adc	285
ADCSRB	0x7B	-	adc	287
ADMUX	0x7C	-	adc	281
TCCR1A / TCCR1B	0x80 / 0x81	-	timer	154, 156
OCR1AL / OCR1AH	0x88 / 0x89	-	timer	159
TCCR4A / TCCR4B	0xA0 / 0xA1	-	pwm	154, 156
ICR4L / ICR4H	0xA6 / 0xA7	-	pwm	161
OCR4AL-H / OCR4BL-H / OCR4CL-H	0xA8-0xA9 / 0xAA-0xAB / 0xAC-0xAD	-	pwm	159-160
TCCR5A / TCCR5B	0x120 / 0x121	-	ultra	154, 156
TCNT5L / TCNT5H	0x124 / 0x125	-	ultra	158

Bits used

Register	Bits used by the drivers
SREG	I (7): global interrupt enable
TCCR0A / TCCR0B / TIMSK0	WGM01 (1): CTC · CS02 (2): clk/256 · OCIE0A (1): compare A interrupt
TCCR1B / TIMSK1	WGM12 (3): CTC · CS11 (1) + CS10 (0): clk/64 · OCIE1A (1)
TCCR4A	COM4A1 (7), COM4B1 (5), COM4C1 (3): non-inverting output · WGM41 (1)
TCCR4B	WGM43 (4) + WGM42 (3): mode 14 with WGM41 · CS41 (1): clk/8
TCCR5B	CS51 (1): clk/8, normal mode
ADMUX	REFS0 (6): AVcc reference · MUX4..0 (4..0): channel
ADCSRA	ADEN (7): on · ADSC (6): start / busy · ADPS2..0 (2..0) = 111: clk/128
ADCSRB	MUX5 (3): channels 8..15

12 Interrupts and 16-bit Access

Interrupt vectors

Without `avr-libc` there is no `ISR()` macro. A handler is a normal function with the name `__vector_N`, where N is the datasheet vector number minus 1: the datasheet counts from 1 (RESET is No. 1), `avr-gcc` counts from 0. The `signal` attribute makes the compiler save every register it uses (and `SREG`) and return with `RETI`; `used` and `externally_visible` stop the compiler from removing it.

Source	Datasheet No.	Word address	Handler	Driver	Work done
TIMER1 COMPA	18	\$0022	<code>__vector_17</code>	timer	Adds 1 to the millisecond counter
TIMER0 COMPA	22	\$002A	<code>__vector_21</code>	seg7	Shows the next digit

Example

```
void __vector_17(void) __attribute__((signal, used, externally_visible));
void __vector_17(void)
{
    timer_ticks++;
}
```

Global interrupt enable

Interrupts are turned on by setting bit I in `SREG` (`M2560_SREG |= (1 << 7)`), which is what `sei()` does. `timer_init` and `seg7_init` do this. When the 32-bit millisecond counter is read, `SREG` is saved, bit I is cleared, the counter is copied and `SREG` is written back.

16-bit registers

`OCR1A`, `ICR4`, `OCR4x`, `TCNT5` and `ADC` are 16-bit registers reached through two 8-bit addresses that share a hidden `TEMP` byte (datasheet section 17.3, "Accessing 16-bit Registers").

- **Write:** high byte first, then low byte. Writing the low byte copies both bytes in at the same moment.
- **Read:** low byte first, then high byte. For the `ADC`, reading `ADCL` locks the result until `ADCH` is read.

The compiled code was checked with `avr-objdump` to keep this order.

13 Memory Footprint and Build

Code and data size

Measured with `avr-size` on each driver object, `avr-gcc 15.2`, `-mmcu=atmega2560 -Os`. RAM is static data (`.bss` + `.data`); the `seg7` segment table (18 bytes) is counted in flash here and also copied to RAM at start-up.

Driver	Flash (bytes)	RAM (bytes)
gpio	384	0
led	214	17
sw	322	37
timer	380	5
seg7	696	12
ir	232	13
ultra	648	17
pwm	520	6
adc	260	0
Total	3656	107

A complete test program with all drivers linked and the `avr-gcc` start-up code is about 4.2 KB of the 256 KB flash (1.6 %).

Build

Item	Value
Compiler flags	<code>-mmcu=atmega2560 -DF_CPU=16000000UL -Os -Wall -Wextra</code>
Include paths	<code>-Idrivers -Idrivers/<name></code>
Build a test	<code>make TEST=t_seg7</code>
Flash a test	<code>make flash TEST=t_seg7 [PORT=/dev/ttyUSB0]</code>
Programmer	<code>avrdude -p m2560 -c wiring -b 115200</code>
Fedora packages	<code>avr-gcc avr-libc avrdude make</code>

Folder layout

<code>drivers/regs.h</code>	register addresses
<code>drivers/board.h</code>	wiring of one project (not used by drivers)
<code>drivers/<name>/</code>	<code><name>.c</code> and <code><name>.h</code> for each driver
<code>tests/</code>	one test program per driver
<code>app/main.c</code>	application

14 Reference Wiring (Smart Parking Project)

The drivers do not depend on this wiring. It is the connection used in the parking project, defined in `drivers/board.h` and passed to the init functions by the test programs.

Part	Board pin	Chip pin	Init call
LED SAFE / CAUTION / WARNING / STOP / OCCUPIED	D22 - D26	PA0 - PA4	<code>led_init('A', 0..4)</code>
Push switch to GND	D2	PE4	<code>sw_init('E', 4, SW_ACTIVE_LOW)</code>
7-seg segments a..g, dp	D37 - D30	PC0 - PC7	<code>seg_port = 'C'</code>
7-seg digit 1 (tens)	D41	PG0	<code>d1 = 'G', 0</code>
7-seg digit 2 (ones)	D40	PG1	<code>d2 = 'G', 1</code>
Ultrasonic TRIG / ECHO	D47 / D48	PL2 / PL1	<code>ultra_init('L', 2, 'L', 1)</code>
IR sensor OUT	D46	PL3	<code>ir_init('L', 3, IR_ACTIVE_LOW)</code>
Buzzer	D6	PH3 (OC4A)	<code>pwm_init('A')</code>
Potentiometer	A0	PF0 (ADC0)	<code>adc_read(0)</code>

Project settings in `board.h`: `SEG7_TYPE = SEG7_CATHODE`, `SEG7_DIGIT_ON = 1` (digits switched by NPN transistors), `IR_TYPE = IR_ACTIVE_LOW`.

Test programs

Test	Shows
<code>t_led_sw</code>	LEDs on one by one, then STOP LED follows the switch
<code>t_timer</code>	SAFE LED toggles every 1000 ms
<code>t_seg7</code>	Counts 0..99 every 300 ms
<code>t_ir</code>	OCCUPIED LED follows the IR sensor
<code>t_ultra</code>	Distance in cm on the display every 100 ms
<code>t_pwm</code>	Buzzer at 25 / 50 / 75 % duty, 1 s each
<code>t_adc</code>	Potentiometer value / 11 on the display
<code>t_gate</code>	Occupied: dash + LED, else distance

15 Revision History

Rev	Date	Changes
1.0	08 Oct 2026	First release. Covers the general drivers (pins passed to init, ids for LED/switch/IR/ultrasonic). Source commit 9be096a .

Status

All drivers and test programs build with avr-gcc using `-Wall -Wextra` and zero warnings. The library has not yet been tested on hardware; values in this document come from the source code and the ATmega2560 datasheet.

Authors

Ponmudi. GPIO driver with contributions from Pranesh and Kevin.